



★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Netflix TechBlog · [Follow publication](#)

Democratizing Machine Learning at Netflix: Building the Model Lifecycle Graph

14 min read · May 4, 2026



Netflix Technology Blog

Follow



Listen



Share

... More

Repost to your network. A new and easy way to share your recommended stories with your followers.

Okay, got it

[Saish Sali](#), [Nipun Kumar](#), [Sura Elamurugu](#)

Introduction

As Netflix has grown, machine learning continues to support our ability to deliver value to members and drive excellence across multiple areas of our business. When Netflix began investing in machine learning over a decade ago, it was primarily focused on a single domain: personalization. Scala was the industry standard, our ML teams were relatively small, and optimizing member engagement was our primary use case. Fast forward to today, and machine learning has become the backbone of Netflix's business transformation. We now apply ML across various business domains, including:

- **Personalization:** Optimizing engagement and helping members discover content they'll love
- **Studio:** Pre and post-production workflows
- **Payments:** Fraud detection, payment routing, and recurring billing optimization
- **Ads:** Our newest domain, requiring real-time decisioning and targeting

... and a growing number of additional use cases across the company

Each domain operates with a different tech stack, different business metrics, and a distinct organizational structure. While this diversity is a testament to how machine learning has evolved to drive value across many verticals at Netflix, this growth introduces a new challenge: **enabling cross-pollination of models and data across domains.**

The Challenge: A Fragmented ML Landscape

As our ML investments scaled across these domains, a critical problem emerged: the models produced largely became black boxes. Without any discovery infrastructure, ML practitioners couldn't easily collaborate or share work across business verticals.

Consider a concrete example: content embeddings. Our Studio teams create sophisticated embeddings that identify scene boundaries, detect visual transitions, and understand content structure. These embeddings were originally built for production workflows.

But those same embeddings could be incredibly valuable elsewhere. Ads could hypothetically use content embeddings for context matching (ensuring advertisements align with the tone and content of what's currently playing). Personalization could leverage them for episodic merchandising and recommendations (matching the topic or mood of an episode with a user's preferred viewing preferences). Yet making this cross-pollination happen is extraordinarily difficult.

Why? Our ML tools exist in silos, each with its own backend services and user interface. The model registry is unaware of which A/B tests were using its models, and the pipeline orchestrator is unaware of downstream model dependencies. ML practitioners have to traverse multiple systems to answer basic questions about their work. Finding a model requires opening the model registry, understanding its lineage means switching to the pipeline orchestrator, and tracking which A/B tests use that model requires navigating to the experimentation platform. This fragmentation prevents practitioners from answering critical questions:

- **Discovery:** What features exist? What data sources are available for generating features for a model?

- **Lineage:** Which pipeline is generating data for a specific model? What data sources feed those features?
- **Impact:** Which A/B tests are running this model? Which models will break if I change this feature? Who owns each piece of this chain?

The Hard Problem: Connecting everything

The real challenge wasn't just building a consolidated UI. We needed to connect the different pieces of infrastructure our ML practitioners were using to perform different parts of the ML lifecycle.

Our ML ecosystem generates metadata from dozens of sources:

- Pipeline orchestration systems emit execution details, stage dependencies, and data transformations
- Deployed model registry tracks model versions, artifacts, staleness, and deployment history
- Experimentation platform manages A/B tests and their configurations
- Feature store catalog feature definitions and usage
- AI Dataset platform tracks the creation, management, discovery, and loading of datasets.
- Identity platform maintains user, team, and organization metadata

Each system employs different formats, identifiers, and mental models. The hard technical problem we had to solve was: **How do we collect this heterogeneous metadata, transform it into a unified entity model, and build a connected graph that enables true exploration and collaboration across business domains?**

The Solution: Metadata Service and the Model Lifecycle Graph

Our answer was the Metadata Service (MDS), which builds a Model Lifecycle Graph that indexes and connects ML-related entities across Netflix. MDS is optimized for real-time ingestion of ML metadata (e.g., models, features, pipelines, experiments, datasets) and to answer cross-domain questions such as “Which experiments are running this model?” or “Which models share these features?” It is the foundation that enables discovery, ingesting events from diverse sources, enriching them with context, and materializing relationships across entities.

Our vision: to make every ML asset at Netflix discoverable, understandable, and reusable by every ML practitioner, regardless of their team or domain.

Core Abstractions: The Vocabulary of the System

Before diving into the technical implementation, it's helpful to understand the conceptual model that underpins MDS. This vocabulary enables consistent communication across teams and systems:

Component: Any object that is uniquely addressable using an AI Platform's (AIP) Uniform Resource Identifier (URI). An AIP URI follows the format `aip://<componentType>/<platformId>/<resourceId>` , ensuring global uniqueness. For example:

- **Models:** `aip://model/registry/ranking-v5`
- **Users:** `aip://user/identity/alice`
- **Pipelines:** `aip://pipeline/orchestrator/weekly-training`

Entity: A component within the ML ecosystem, characterized by additional properties such as name, description, creation date, and owners. Entities represent ML-specific assets, such as models, features, and pipelines.

Entity Type: A group of entities that share the same data shape. A data shape is a set of property constraints that specify the attributes and relationships an entity must have.

Domain: A functional grouping of related entity types that defines the abstract interface for a category of ML assets. For example, the Models domain defines what a Model and Model Instance look like, while the Pipelines domain defines Schedules, Requests, and Executions.

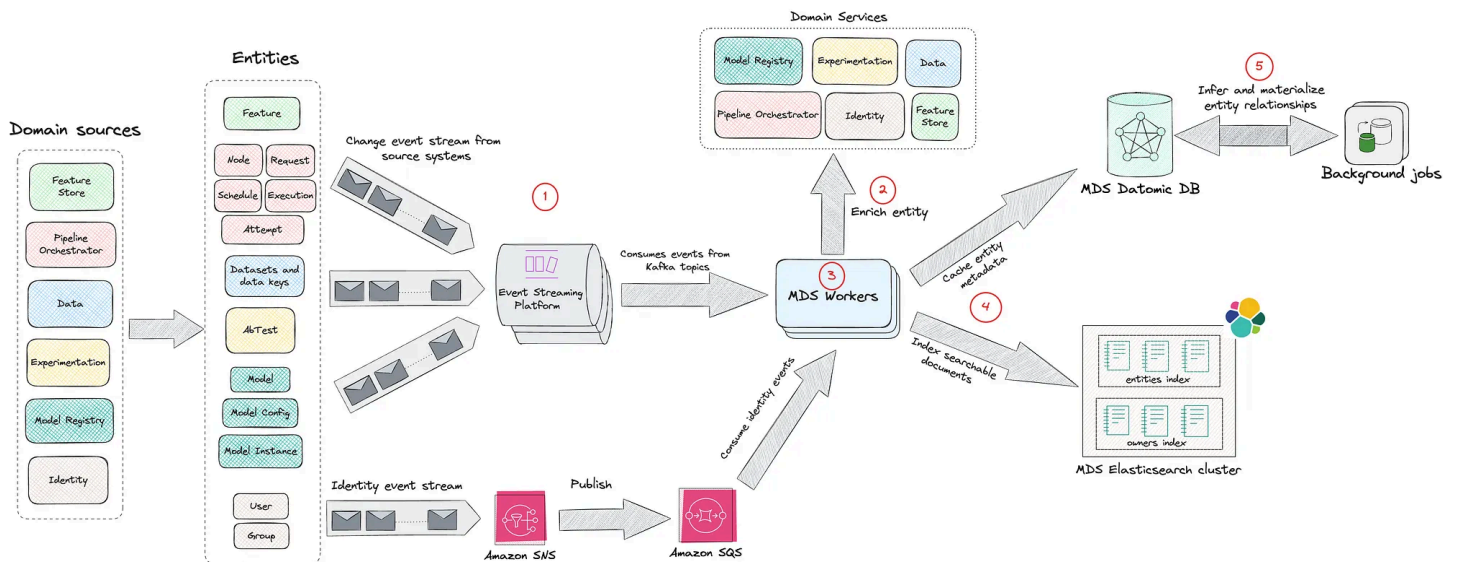
Provider: A concrete implementation of a domain, backed by a specific source system. For example, the Models domain is currently backed by our internal model registry. This separation allows MDS to support multiple providers for the same domain. If a new model registry were introduced, it could be added as an additional provider without changing the domain interface.

We can summarize these concepts with a concrete example:

Concept	Definition	Example(s)
Component	Addressable object with an AI Platform's (AIP) URI	aip://model/registry/ranking-v5, aip://user/identity/alice
Entity	Component with ML metadata (name, owners, tags, etc.)	A Model named "ranking-v5" owned by the Personalization team
Entity Type	Schema/data shape shared by a set of entities	Model, Feature, PipelineRun, Dataset, A/B Test
Domain	Functional grouping defining abstract entity interfaces	Models (Model, ModelInstance), Pipelines (Schedule, Request, Execution), Experimentation (A/B Test, A/B Cell)
Provider	Concrete implementation of a domain backed by a source system	Model Registry (for Models domain), Pipeline Orchestrator (for Pipelines domain)

This URI-based addressing scheme is crucial as it allows any service to reference any ML asset with a single string, and MDS can resolve that reference back to rich, connected metadata.

From Events to Entities to Graph



The journey from raw system events to a queryable graph happens in stages. Let's walk through each with a concrete example: connecting a model to its A/B tests through relationship inference.

1 Event Ingestion

MDS integrates with various source systems via Kafka and AWS SNS/SQS, consuming events in real-time. Source systems emit thin events that include an identifier and

an event type.

Example event:

```
{
  "event_type": "model_instance_created",
  "instance_id": "ranking-model-v5-20XX0101",
  ...
}
```

This design keeps producers simple. Source systems only need to announce that a change occurred, without building complete payloads or understanding downstream requirements.

Each source system has dedicated event handlers in MDS:

- **Pipeline Orchestration:** Ingests pipeline execution events, including node definitions, schedules, requests, and job attempts
- **Model Registry:** Captures model deployments, configurations, and version updates
- **Feature Store:** Tracks feature definitions and their versions
- **Experimentation Platform:** Monitors A/B test configurations and allocations
- **Datasets:** Tracks ML datasets and their versions
- **Identity Platform:** Maintains ownership and team membership information

2 Entity Enrichment

MDS implements a hydration contract for each event type. When an event arrives, MDS:

1. Validates the event schema
2. Calls the source system's API to fetch the complete, current state
3. Transforms the response into a normalized entity

This design has a crucial property: the order of events doesn't matter. MDS always fetches the latest facts from the source of truth. This pattern decouples the event stream from state consistency. If the event bus drops a message or delivers it out of order, the next event corrects the state. The event stream becomes a notification of change rather than a log of changes.

This notification of change pattern has a few important tradeoffs. On the plus side, it keeps producers simple, makes us robust to out-of-order or dropped events, and ensures that MDS can always reconcile to the latest state by reading from the source of truth. The tradeoff is that we place additional read load on source systems during hydration and need to be deliberate about rate limiting, caching, and backoff in our enrichment workers so that we don't overload them.

For our ranking model example, when the `model_instance_created` event arrives, MDS calls the Model Registry API: `GET /api/v1/instances/ranking-model-v5-20XX0101`

The registry responds with a full descriptor. Example response (key fields only):

```
{
  "id": "ranking-model-v5-20XX0101",
  "pipeline_run_id": "train-weekly-ranking-20XX0101",
  "owner_emails": ["alice@netflix.com"],
  "labels": [{"key": "team", "value": "personalization"}],
  ...
}
```

3 Data Transformation and Normalization

Raw events are heterogeneous and each source system has its own schema and semantics. MDS workers transform these events into a unified entity model with standardized fields.

Without normalization, downstream consumers would need to understand every source system's schema. Normalization creates a consistent interface, allowing queries and relationships to work across all entity types. Here is an example.

Normalized MDS entity:

```
{
  "id": "aip://model/registry/ranking-model-v5-20XX0101",
  "pipeline_run": "aip://pipeline-run/orchestrator/train-weekly-ranking-20XX0101",
  "entity_type": "ModelInstance",
  "owners": ["aip://user/identity/alice"],
  "tags": [{"tag": "team", "value": "personalization"}],
  ...
}
```

The normalization process standardizes field names and formats. For example, platform-specific IDs become global AIP URIs, owner_emails becomes owners with resolved user URIs, and labels become tags. Foreign keys like pipeline_run_id are transformed into entity references. However, there's still no reference to which A/B tests are using this model. The Model Registry doesn't track experiments, and the Experimentation Platform doesn't track which pipeline produced a given model. This is where knowledge enrichment becomes critical.

4 Storage and Indexing

Once normalized, entities are persisted to Datomic and immediately indexed in Elasticsearch. This happens synchronously within the event processing flow.

Datomic for Caching and Relationships

Normalized entities are first written to Datomic, which serves as both a local cache and a graph database.

Why Datomic? Datomic serves as both the system of record for MDS and the working dataset for enrichment processes. Its immutable fact model means we can continuously add relationships without losing the original entity state.

What we store:

- All entity attributes as facts
- Entity references (foreign keys that may point to entities not yet fully resolved)
- All relationships as reified edges (added by enrichment processes)
- Entity lifecycle state (tracking which entities are fully enriched vs awaiting hydration)

This enables:

- **Complex graph traversals:** Navigate from a model to its features to their data sources in a single query
- **Entity relationships:** Join across multiple domains without N+1 query problems
- **Flexible schema evolution:** Easy to add new entity types and attributes as the catalog grows
- **Progressive enrichment:** Background jobs efficiently identify and process entities requiring additional hydration, enabling gradual graph completion without reprocessing fully enriched entities

In practice, we use Datomic for relationship-heavy, navigational queries such as:

- Starting from this model instance, show me all upstream datasets and downstream experiments.
- Given this feature, list all consuming models and their owning teams.

These queries often span multiple hops in the graph and benefit from Datomic's immutable fact model and efficient joins across entity relationships.



Elasticsearch for Discovery

Immediately after writing to Datomic, entities are indexed in Elasticsearch to power fast, full-text search across the catalog.

What we index:

- **Primary fields:** Entity name, description, entity type, owner names
- **Relationship metadata:** Names of related entities (e.g., a model's features, pipelines, A/B tests) stored in the related field

- Tags: Domain-specific metadata stored as key-value pairs (e.g., *team::personalization*, *env::production*, *model.state::released*)

Index structure:

- Single entities index: All entity types (models, features, pipelines, etc.) are indexed in one unified index, differentiated by the `entityType` field
- Separate owners index: Dedicated index for users and groups to enable cross-entity owner searches
- Relevance boosting: Exact name matches score higher than other relevant matches

This enables:

- Multi-field text search across entity names, descriptions, tags, and related metadata
- Relevance ranking with boosting (exact name matches score significantly higher)
- Complex filtering by entity type, ownership, tags, and domain-specific attributes (stored as tags)
- Fuzzy matching to handle typos and partial queries

Elasticsearch powers the entry point into the system: users typically start with a free-text search in the AIP Portal (for a model name, a team, or a domain term), and then switch to graph navigation once they land on an entity page. Indexing happens in near real-time as part of the ingestion and enrichment workflows, so changes are usually visible in the Portal with a short delay that is acceptable for interactive use.

5 Knowledge Enrichment and Graph Formation

Once entity metadata is persisted in Datomic, scheduled background processes take over to discover and materialize relationships. These enrichment jobs run periodically, scanning for uncached or partially resolved entities (entities that exist only as references without full metadata).

The enrichment workflow:

- **Identify candidates:** Find entities marked as uncached or with unresolved references
- **Hydrate relationships:** Query source-of-truth systems to fetch related entity details
- **Materialize edges:** Write discovered relationships back to Datomic
- **Re-index:** Trigger Elasticsearch indexing for updated entities
- **Mark as enriched:** Update entity status to prevent redundant processing

This asynchronous approach allows MDS to handle the computational cost of graph formation without blocking real-time event ingestion. It also enables retry logic and gradual enrichment as new entities become available.

Because enrichment is asynchronous, newly discovered relationships may appear with a short delay after the underlying entities are created (typically minutes rather than seconds). We track when each entity was last enriched and surface this timestamp in the AIP Portal, so practitioners can reason about staleness and know when it's safe to rely on a particular relationship for debugging or impact analysis.

Why enrich? Source systems are purpose-built and don't know about entities in other domains. Enrichment discovers and materializes cross-system relationships that enable powerful lineage and impact queries.

Example: Connecting Models to A/B Tests

When MDS processes a new model instance, background enrichment jobs discover relationships through multi-hop inference:

Step 1: Direct link to pipeline

The model references a `pipeline_run_id`. An enrichment job hydrates the pipeline and discovers its A/B test associations: `GET /api/v1/pipeline-runs/train-weekly-ranking-20XX0101`

Response:

```
{
  "run_id": "train-weekly-ranking-20XX0101", "pipeline": "weekly-ranking-trainer"
  "ab_test_cells": [
```

```
    {"test_id": "12345", "cell_number": 2, "cell_name": "treatment_ranking_v5"}  
  ]  
  ...  
}
```

Step 2: Discover A/B test context

The enrichment job discovers the pipeline ran for A/B test cell #2 and queries the Experimentation Platform for test details: `GET /api/v1/tests/12345`

```
{  
  "test_id": "12345",  
  "name": "Ranking Model v5 vs v4",  
  "status": "ACTIVE",  
  "cells": [{"cell_number": 1, "name": "control_ranking_v4"}],  
  ...  
}
```

Step 3: Infer transitive relationships

The enrichment job now has the complete chain:

- Model Instance was produced by Pipeline Run
- Pipeline Run was executed for A/B Test Cell #2
- The A/B Test Cell #2 belongs to A/B Test “Ranking Model v5 vs v4”
- Model Instance now gets associated with this A/B Test

The job writes the inferred relationship back to Datomic and triggers re-indexing, and materializes these edges in the graph. MDS doesn’t just store what it’s told; it derives new knowledge by *walking* the graph in the background.

Why this matters: Without MDS, answering “Which A/B tests are using this model?” requires:

1. Looking up the model in the Model Registry
2. Finding which pipeline produced it

3. Checking the Pipeline Orchestrator for A/B test tags

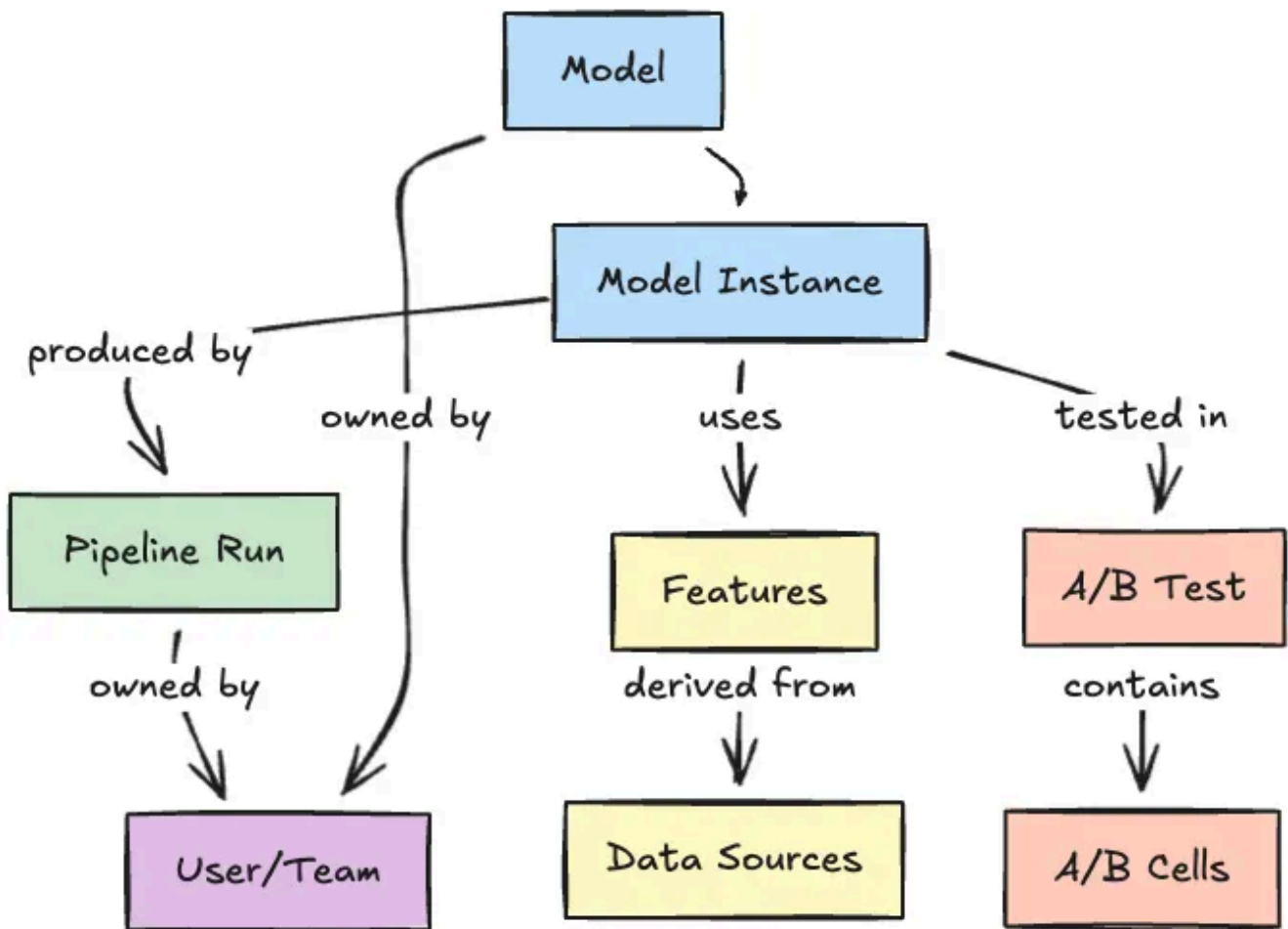
4. Querying the Experimentation Platform for test details

With the model lifecycle graph, it's a single query:

```
query {  
  model(id: "aip://model/registry/ranking-model-v5-20XX0101") {  
    name  
    owners { name }  
    currentInstance {  
      version  
      pipeline {  
        name  
        owners { name }  
      }  
      features {  
        edges {  
          node {  
            name  
            data { edges { node { name } } }  
          }  
        }  
      }  
    }  
    associatedAbTests {  
      name  
      cells { number name }  
    }  
  }  
}
```

The reverse query also works: “What models are being tested in experiment 12345?”

Enabling Exploration, Not Just Search



With the Model Lifecycle Graph in place, we shift from entity search to entity exploration. Discovery isn't just about finding a model; It's about traversing relationships:

- Start with a model, explore its features
- From features, navigate to the core data driving them
- From the data, trace back to the pipelines generating it
- From pipelines, see which teams own and depend on them
- From experiments, understand which models are being tested

For example, imagine an engineer investigating a degraded engagement metric for a personalization model. They might:

1. Start with the model instance powering the affected recommendations in the AIP Portal.
2. Inspect the model's features and follow a suspicious feature to its upstream dataset.

3. From the dataset page, see that its pipeline recently had failed runs and identify the owning team.
4. Confirm which A/B tests are currently running this model instance to understand which members and surfaces are impacted.

Before MDS and the Model Lifecycle Graph, this required manual checks across multiple tools (model registry, pipeline orchestrator, experiment platform). Now it's a contiguous journey in a single interface.

This graph-based exploration answers questions that were previously impossible:

- Lineage queries: What is the complete lineage of this model, from training data to production experiments?
- Impact analysis: Which models will be affected if I change this feature?
- Usage discovery: Which A/B tests are using this model?
- Dependency mapping: What data sources does my pipeline transitively depend on?
- Deprecation planning: Which entities are no longer being used and can be retired?

Every entity has deep context: its creation time, ownership, update history, and most importantly, its relationships to other entities.

The Model Lifecycle Graph is surfaced to practitioners through the AIP Portal, a unified interface that provides full-text search across all entity types, detailed entity pages with navigable relationships, and personalized views for teams and individuals.

A typical interaction in the AIP Portal looks like:

- **Search:** Type a model, feature, dataset, or team name into the single search box backed by Elasticsearch.
- **Inspect:** Land on an entity page that shows key metadata (description, owners, domains, tags) alongside a relationships panel.

- **Explore:** Click through to related entities (upstream datasets, downstream experiments, and sibling model versions) to navigate the Model Lifecycle Graph without leaving the portal.

When new entity types are introduced into MDS, the portal automatically provides baseline search, entity pages, and relationship navigation, and we can then layer on domain-specific visualizations (such as model deployment history or dataset version timelines) over time.

The Road Ahead: Open Challenges

Building the ML lifecycle graph is an ongoing journey. Significant challenges remain, and these represent the future opportunities for us:

- **Tool Proliferation:** As new ML tools emerge, we need robust integration patterns that scale. How do we design plugin architectures that make adding new sources seamless? If we don't keep up with new tools, practitioners will be forced back into fragmented views, and the Model Lifecycle Graph will lose coverage and trust.
- **Domain-Specific Visualizations:** Different entity types require distinct visualization experiences. Model pages should display deployment history, A/B test associations, and performance metrics. Feature pages should highlight data lineage and consuming models. Pipeline pages must show execution history, dependencies, and schedules. Dataset pages require versioning timelines and downstream consumers. How do we design a flexible UI framework that allows each entity type to have its own tailored experience while maintaining consistent navigation and interaction patterns across the portal? Without rich, domain-specific experiences, the portal risks becoming a generic catalog rather than a tool that ML practitioners rely on in their daily workflows.
- **Metadata Quality:** Today, MDS ensures data consistency through source-of-truth hydration and schema validation at ingestion. Background enrichment jobs continuously infer relationships and materialize entities from source systems. However, challenges remain in ensuring completeness and timeliness at scale. When source systems fail to emit events, when ownership information becomes stale, or when entities lack descriptions and contextual metadata, the graph's utility degrades. How do we build automated validation and enrichment systems to detect metadata anomalies, suggest missing relationships, and maintain quality benchmarks across millions of entities? Poor or stale metadata erodes

practitioner trust: if the graph is incomplete or incorrect, teams will revert to ad hoc knowledge and one-off integrations rather than using MDS as their source of truth.

- **Advanced Relationship Inference:** Beyond explicit relationships declared in source systems, how do we infer implicit connections? Can we detect that two models serve similar purposes based on shared features? Can we recommend features based on usage patterns from similar pipelines? We are in the early stages of exploring these ideas. Done well, they would turn MDS from a passive catalog into an active recommendation engine for ML assets, accelerating reuse and reducing duplicate work across domains.

Acknowledgments

This work represents the collective effort of stunning colleagues across the AI Platform organization: [Emma Carney](#), [Megan Ren](#), [Nadeem Ahmad](#), [Pat Oleniuk](#), [Prateek Agarwal](#), [Tigran Hakobyan](#), [Yinglao Liu](#)

Machine Learning

Mlops

Distributed Systems

Event Driven Architecture

Knowledge Graph



Follow

Published in Netflix TechBlog

184K followers · Last published 1 day ago

Learn about Netflix's world class engineering efforts, company culture, product developments and more.



Follow

Written by Netflix Technology Blog

458K followers · 1 following

Learn more about how Netflix designs, builds, and operates our systems and engineering organizations
